

Multiprocessor architecture evolution

Below is a **fully detailed, well-structured, clearly formatted explanation** of the entire chapter section you provided.

Every concept, motivation, taxonomy, challenge, example, and computation is explained **deeply**, without skipping any point.

4.1 Introduction – Detailed Explanation

Advances in computer architecture have historically been driven by the desire to increase performance. For many years, researchers believed that improvements in **uniprocessor** (single-CPU) systems were approaching a limit. However, performance data showed otherwise:

- Between **1986 and 2002**, uniprocessor performance grew at the fastest rate since the earliest transistorized computers (1950s–60s).
- This growth was primarily due to the rise of the **microprocessor**, with aggressive improvements in clock speed, instruction-level parallelism (ILP), and pipelined architectures.

Yet, despite this impressive growth, a number of trends pushed the industry toward **multiprocessors**:

Why Multiprocessors Became Important

Several technological and application-driven reasons led to an increased dependence on multiprocessors during the 1990s and early 2000s:

1. Server and enterprise computing grew rapidly

Businesses required systems capable of handling simultaneous requests, large databases, and high transaction volumes. Multiprocessors are well-suited for such workloads due to natural parallelism.

2. Emergence of data-intensive applications

Modern workloads—analytics, scientific computation, large-scale simulation, and multimedia—benefit from parallel execution.

3. Diminishing returns in ILP (Instruction-Level Parallelism)

Traditional techniques (deeper pipelines, wider issue superscalars, speculative execution) were providing less benefit, making it harder to scale uniprocessor performance.

4. Increasing concerns over power consumption

High-frequency, complex uniprocessors consume large amounts of power. Multiprocessors allow performance scaling through parallelism instead of higher clock speeds.

5. The desktop performance plateau

Beyond graphics-intensive applications, typical desktop applications do not require extreme single-thread performance.

6. Better understanding of parallel programming models

Server environments naturally contain independent threads (user requests, sessions, transactions) which map well to multiprocessors.

7. Leveraging replication over complex design

It is easier and more cost-effective to duplicate a simple processor core multiple times than to keep building a larger, more complex superscalar core.

Scope of the Chapter

Multiprocessor design is extremely broad. Entire books are devoted to it.

This section focuses on:

- **Mainstream multiprocessor systems:** 4–32 processors
- These systems dominate both sales volume and revenue
- Large-scale multiprocessors (hundreds to thousands of processors) are treated briefly in Appendix H

A Taxonomy of Parallel Architectures

To understand multiprocessor designs, Flynn (1966) proposed a simple and still-useful classification based on:

- Number of instruction streams
- Number of data streams

This gives four categories:

1. SISD — Single Instruction, Single Data

- Standard uniprocessor system
- One instruction stream operates on one data stream
- No parallelism

2. SIMD — Single Instruction, Multiple Data

- One instruction applied simultaneously to many data elements

- Suitable for **data-level parallelism**, e.g., vector processing, matrix operations, image processing.

Characteristics:

- One control unit broadcasts the same instruction to many processing elements
- Each processor has its **own data memory**
- A single instruction memory

Examples:

- Vector processors (Appendix F)
- Multimedia extensions (MMX, SSE)
- Modern GPUs

This approach is extremely efficient for uniform, parallel data operations.

3. MISD — Multiple Instruction, Single Data

- Multiple instructions operate on the same data stream
 - No commercially successful architecture of this type was produced
-

4. MIMD — Multiple Instruction, Multiple Data

- Each processor fetches its own instruction stream and accesses its own data
- Flexible and powerful; supports **thread-level parallelism (TLP)**

This is the dominant architecture for general-purpose multiprocessors.

Reasons MIMD became the standard:

Flexibility

MIMD machines can:

- Run independent processes (multiprogramming)
- Run parallel threads from the same program
- Serve multiple users simultaneously

Cost-effectiveness

They use:

- Commodity processors (same as desktops and servers)
- Replicated cores in multicore designs

Clusters

A popular MIMD design:

- Built using standard processors and network components
- Common for high-throughput servers and scientific applications

Two types of clusters:

1. **Commodity clusters**

- Use off-the-shelf computers and networking
- Easy to build, inexpensive
- Used for web servers, transaction systems

2. **Custom clusters**

- Optimized for parallel scientific workloads
 - Require low-latency, high-bandwidth networks
 - Example: IBM Blue Gene
-

Rise of Multicore Processors

In the 1990s and 2000s, chip density improved enough to allow **multiple processor cores on a single die**:

- Originally called *single-chip multiprocessing*
- Now simply **multicore**

Modern examples include:

- IBM Power5
- Sun T1
- Intel Pentium D / Xeon MP

Multicore achieves performance gains by **replicating simpler cores** rather than building one extremely complex superscalar processor.

Threads and Processes in Multiprocessors

Process

A process is an independent program with its own address space, registers, and execution context.

Thread

Multiple threads can:

- Share code and data within the same process

- Execute independently
- Run on different processors in a multiprocessor system

Threads exploit **thread-level parallelism (TLP)**.

Grain Size

Amount of work per thread:

- Coarse-grained: millions of instructions (better for multiprocessors)
- Fine-grained: small loops (harder due to overheads)

Types of MIMD Multiprocessors: Shared Memory vs Distributed Memory

The classification is based on how memory is organized.

1. Centralized Shared-Memory (Symmetric Multiprocessors, SMP / UMA)

Characteristics:

- A few dozen processors (up to around 32)
- All processors share **one physical memory**
- Equal memory access time → **Uniform Memory Access (UMA)**
- Large shared memory is accessed through:
 - Buses
 - Crossbar switches
 - Multi-bank memory

Advantages:

- Easy to program
- Shared-memory communication is simple: loads and stores

This design is dominant for small to medium multiprocessors.

2. Distributed-Memory Multiprocessors (NUMA)

Characteristics:

- Larger number of processors
- Memory is physically distributed across nodes

- Each node contains:
 - Processor(s)
 - Local memory
 - I/O subsystem
 - Network interface

Benefits:

- Higher memory bandwidth (each node has its own memory)
- Lower local memory latency
- Scales better for large systems

Drawbacks:

- Communication is more complex
- Software must ensure data locality

Communication Models

Two models exist depending on the memory layout:

1. Shared-Address-Space Model (Distributed Shared Memory - DSM)

- processors share a single logical address space
- same address refers to the same memory location across processors
- access time is non-uniform → **NUMA** architecture
- communication through loads and stores

2. Message-Passing Model

- no shared address space
- each processor has its own private memory
- communication occurs explicitly using messages (send/receive)
- used in clusters and many large-scale HPC systems

Challenges in Parallel Processing

There are two fundamental problems:

1. Limited Parallelism (Amdahl's Law Problem)

Parallel systems can only speed up the parallel portion of a program.
If even a small fraction remains sequential, speedup is severely limited.

Example

Desired speedup = 80

Processors = 100

We calculate how much of the program must be parallelizable.

Result:

Only **0.25%** of the computation can be sequential.

Meaning:

- To get near-linear speedup, sequential portions must be extremely small.
- Real programs rarely achieve such low sequential fractions.

2. Long-Latency Remote Communication

Accessing remote memory on another processor is expensive:

- Multicore: ~50 cycles
- Large multiprocessor: ~1000+ cycles

This stalls the processor and reduces speedup.

Example: Communication Overhead Impact

System: 32-processor multiprocessor

Remote memory access time: 200 ns

Clock: 2 GHz $\rightarrow$ 1 cycle = 0.5 ns $\rightarrow$ 200 ns = 400 cycles

Base CPI (local cache hits): 0.5

Remote access frequency: 0.2% of instructions

Remote stall cycles per instruction:

$$= 0.002 \times 400 = 0.8$$

Total CPI:

$$= 0.5 + 0.8$$

$$= \mathbf{1.3}$$

If all accesses were local:

CPI = 0.5, so speedup due to perfect locality:

$$1.3 / 0.5 = \mathbf{2.6\times \text{faster}}$$

This shows how severely remote access latency can degrade performance.

How to Address These Challenges

1. Improve parallelism in software

- Better algorithms
- Improved load balancing
- Finer decomposition of tasks

2. Reduce or hide communication latency

Hardware techniques:

- Cache shared data
- Use coherence protocols (Section 4.2 and 4.3)
- High-speed interconnects

Software techniques:

- Data restructuring to improve locality
- Reduce shared memory accesses
- Latency hiding with multithreading
- Prefetching

3. Manage synchronization overhead

- Avoid excessive locking
- Use scalable synchronization primitives (Section 4.5)

4. Adopt appropriate consistency models

- Memory consistency rules define how memory updates appear across processors
- Section 4.6 covers this in detail

Prerequisites

A solid understanding of cache memory is required:

- Write-back vs write-through caches
- Multi-level caches
- Cache coherence concepts

If these are unfamiliar, Appendix C must be reviewed first.

Summary of Key Points

- Uniprocessor performance slowed due to ILP saturation and power issues.
- Multiprocessors offer a scalable path to higher performance.
- Flynn's taxonomy categorizes architectures into SISD, SIMD, MISD, MIMD.
- MIMD is dominant for general-purpose systems due to flexibility and cost effectiveness.
- Two memory architectures exist:
 - Centralized shared-memory (SMP/UMA)
 - Distributed memory (NUMA)
- Communication models:
 - Shared memory
 - Message passing
- Parallel computing is limited by:
 - Lack of parallelism (Amdahl's Law)
 - High communication cost
- Techniques exist to reduce latency, improve locality, and optimize synchronization.

If you want, I can also create:

- A **diagram-based summary**
- A **short exam-oriented version**
- A **longer textbook-style rewritten chapter**
- **Practice questions** with solutions
- A **PPT with slides for this section**

Just tell me what you want next.

I'll walk through this section in a clean structure, and I'll restate *all* important ideas in your text, including the subtle protocol and implementation details. I'll also "verbally reconstruct" the figures where needed.

4.2 Symmetric Shared-Memory Architectures – Detailed Explanation

1. Basic Idea of Symmetric Shared Memory (SMP / UMA)

1.1 Role of Caches

Modern processors use **large, multilevel caches** (L1, L2, often L3) to reduce the pressure on main memory:

- Caches store frequently used data close to the CPU.
- This drastically **reduces the memory bandwidth demand** of a *single* processor.

Because of this reduction:

- If each processor's main memory demand is smaller (thanks to caches),
- Then **multiple processors** can share **the same main memory** without saturating the memory bus.

1.2 Early Bus-Based Multiprocessors

Starting in the **1980s**, with microprocessors becoming dominant:

- Designers built *small-scale multiprocessors* using:
 - Several processors
 - A **single shared physical memory**
 - Connected by a **shared bus**
- Caches significantly reduced required bus bandwidth, so:
 - These symmetric multiprocessors (SMPs) were **very cost-effective**, as long as memory bandwidth was sufficient.

Implementation evolution:

- Early designs:
 - Each processor + its cache were on a **board**.
 - The board plugged into a **bus backplane** shared with other boards.
- Later (1990s):
 - Higher density: **2-4 processors per board**
 - Used **multiple buses** and **interleaved memories** to support faster processors and higher bandwidth.

1.3 Emergence of On-Chip Multiprocessors (Multicore)

Milestones:

- **2000:** IBM introduces the first **on-chip multiprocessor** for general-purpose computing.
- **2005:** AMD and Intel release **dual-processor** (2-core) versions for the server market.
- **2006:** Sun introduces **T1**, an 8-core multicore chip.

(Section 4.8 of the book discusses T1's design and performance.)

As processors got faster:

- Their **memory demands** also increased.
 - At some point, demands **exceeded what a reasonable shared bus can support**.
 - Result: modern designs often use:
 - Small-scale **switches** or
 - Limited **point-to-point networks** instead of a single bus.
-

2. Caching Private vs Shared Data

Symmetric shared-memory machines usually cache **both**:

1. Private data

- Used by only one processor.
- When cached:
 - The data is **migrated** into the cache.
 - Access is faster (lower latency).
 - Memory bandwidth usage decreases.
- No correctness issue:
 - No other processor uses it.
 - Behavior is identical to uniprocessor behavior.

2. Shared data

- Accessed by **multiple processors**.
- Acts as a form of inter-processor communication through loads/stores.
- When cached:
 - Same data block may be **replicated** across multiple caches.
 - Benefits:
 - Lower access latency.
 - Less memory bandwidth.
 - Less **contention** on shared memory if multiple processors read it simultaneously.

However, caching shared data introduces a **new problem**:

Different processors may hold different, stale values of the *same* memory location in their caches.

This is the **cache coherence problem**.

3. What Is Multiprocessor Cache Coherence?

3.1 The Problem

Imagine:

- A memory location X with value 1.
- Two processors A and B, each with their own cache.
- A writes $X = 0$, but B still has $X = 1$ in its cache.

If B reads X from its cache, it sees **1**, not the new value **0**.

Now A and B disagree on the value of $X \rightarrow$ memory is **incoherent**.

This is exactly what Figure 4.3 illustrates.

3.2 Intuitive Informal Definition

Informally:

A memory system is coherent if any read of a data item returns the **most recently written** value of that item.

This is intuitive but too vague. Two aspects must be clarified:

1. **Coherence** – which values can a read return, given a sequence of reads/writes to the same *location*.
2. **Consistency** – when a write becomes visible to other processors, especially relative to *other memory locations*.

Here, 4.2 focuses on **coherence**; memory consistency is covered later (Section 4.6).

3.3 Formal Conditions for Coherence

A memory system is **coherent** if all three of these hold:

1. Program-order self-read property

A read by processor **P** of location **X** that follows a write by **P** to **X**, with **no other writes to X** in between, must return the value written by **P**.

- This just preserves **program order** on a single processor.

2. Read sees sufficiently separated remote write

A read by some processor of location **X** that follows a write to **X** by *another* processor returns the written value,

if:

- The read and write are sufficiently separated in time, and
- No other writes to **X** occur in between.
- Intuition: if you wait long enough after another processor writes X , you should eventually see that value.

3. Write serialization

All processors must observe **writes to the same location in the same order**.

Example:

- If one processor writes 1 to X, then another writes 2 to X,
- No processor should ever see $X = 2$ and later see $X = 1$.
- Writes cannot “reorder” in time differently for different processors.

This third property is called **write serialization**, and it’s crucial.

Without it, processors might have *contradictory histories* of writes to the same location, which breaks correctness.

3.4 Coherence vs Consistency

- **Coherence**: deals with behavior of reads and writes to **the same memory location**.
- **Consistency**: deals with behavior of operations to **different locations**, and **when** writes become visible globally.

For now, the text assumes two strong ordering properties:

1. A write does **not complete** (and thus does not allow the next write) until **all processors** have seen its effect.
2. The processor does **not reorder writes** with respect to any other memory access.

Consequences:

- If a processor writes A, then writes B,
- Any other processor that sees the new value of B must also see the new value of A.
- Writes are **finished in program order**.
- Reads can still be reordered to some extent.

Later (Section 4.6), the book relaxes these assumptions and discusses **weak/relaxed consistency models**.

4. Basic Schemes for Enforcing Coherence

Multiprocessor cache coherence protocols must support:

- **Migration**: moving data closer to where it is being used (into local caches), reducing latency and remote traffic.
- **Replication**: multiple caches hold copies of read-only or read-mostly shared data, reducing contention and access time.

Instead of avoiding shared-data caching (which would defeat performance benefits), **hardware protocols** are used to preserve coherence:

These are called **cache coherence protocols**.

Key design dimension:

How does the system **track sharing** of each cache block?

Two broad classes:

1. Directory-based protocols

- The sharing status of each memory block is stored centrally in a structure called a **directory**.
- There is one directory entry per memory block.
- Good scalability (used in larger systems).
- Discussed in detail in **Section 4.4**.
- Example: Sun T1 uses a directory-like mechanism.

2. Snooping protocols

- Each cache that has a copy of a block also tracks its **sharing state**.
- No centralized directory.
- All caches are attached to a shared **broadcast medium** (e.g., a bus).
- Every cache controller **snoops** (monitors) all memory transactions on that medium.
- When a cache sees an address it holds, it may respond or update its state accordingly.
- This section (4.2) focuses on snooping protocols.

Snooping fits naturally with **bus-based SMPs**, where a shared bus already exists and is easy to broadcast on.

5. Snooping Protocols

5.1 Two Basic Styles

Two conceptual ways to enforce coherence on writes:

1. Write-invalidate protocol (dominant and important)

- Before a processor writes a cache block, it obtains **exclusive access**:
 - It **invalidates** all other copies of that block in other caches.
- After invalidation:
 - Only the writer has a valid copy.
- All later readers must miss and fetch the **updated** value from memory or the writer's cache.
- This enforces **write serialization**.

2. Write-update (write broadcast) protocol

- On every write to a shared block:
 - The processor **broadcasts the new data** to all other caches that hold that block.
- All caches update their copy.
- Major drawback:
 - Consumes a lot of bandwidth (must broadcast every write).

Because of bandwidth issues, **write-invalidate** protocols are used in essentially all modern multiprocessors.

5.2 How Invalidation Works with a Snooping Bus

Basic mechanism:

- To write to a shared block:
 - The processor must send an **invalidation** on the bus.
- All caches snoop the bus:
 - If a cache has that block, it marks it **invalid**.
- From that point:
 - Only the writer retains a valid copy (exclusive/modified).
 - Any other processor that wants to read the block must miss and fetch the updated data.

This ensures:

- A write followed by another processor's read:
 - The reader cannot use a stale cached copy because it was invalidated.
- Write serialization:
 - If two processors attempt to write the same block at "the same time," they both contend for the bus.
 - The bus arbitration ensures **one winner**:
 - The winner's invalidate runs first.
 - The other processor sees its cached copy invalidated and must refetch the block before writing.

Thus, no two processors can write simultaneously to the same block.

6. Implementation Techniques for Snooping Coherence

6.1 Using the Bus for Invalidations

To invalidate a block in all caches:

1. Processor obtains the bus.
2. Puts the **invalidate request** with the block's address on the bus.
3. All other caches snoop the bus:
 - If they hold that block, they mark it **invalid**.

6.2 Finding the Up-to-Date Value: Write-Through vs Write-Back

When a processor misses in its cache and needs a block:

- **Write-through cache:**
 - Every write is propagated to memory.
 - Memory always has the **latest value**.
 - On a read miss:
 - Memory can simply provide the current block.
 - Simpler for coherence, but more memory traffic.
- **Write-back cache:**
 - Writes update the cache only.
 - Memory may hold a **stale copy**.
 - The most recent copy may be in some cache marked **dirty**.
 - On a read miss:
 - All caches snoop the address.
 - If a cache has the block in modified (dirty) state:
 - It supplies the data block to the bus.
 - Memory's response is canceled or updated.
 - More complex, but greatly reduces bandwidth demand and scales better.

Snooping logic:

- **All caches snoop every memory/bus transaction.**
- If a cache has a **dirty copy**:
 - It *intercepts* the read miss and supplies the fresh data.
- If the block is not dirty anywhere:
 - Main memory can supply it.

6.3 Tracking Sharing: Shared/Exclusive/Owner

To know whether a block is shared or exclusive, each cache block holds extra bits:

- Besides **valid** and **dirty**, add at least one **shared bit**:
 - If shared bit = 1:
 - The block may be present in multiple caches.
 - A write must be preceded by an **invalidate**.
 - If shared bit = 0 and the block is valid:
 - The block is **exclusive** in this cache.
 - Writes can be done locally without broadcasting.

Terminology:

- Cache with the **sole valid copy** of a block is the **owner**.

- Some protocols also have explicit **owner** state (e.g., MOESI):
 - Block may be shared, but one cache is responsible for writing back to memory when needed.

6.4 MESI/MOESI Extension

The basic three states (Invalid, Shared, Modified) can be extended:

- **MESI**: Modified, Exclusive, Shared, Invalid
 - **Exclusive (E)**: block is unmodified and only in one cache.
 - Benefit:
 - If the processor writes to a block in E state:
 - No bus transaction needed (no other cache has it).
 - Just change E → M (modified).
- **MOESI**: Modified, Owned, Exclusive, Shared, Invalid
 - **Owned (O)**: block may be shared, but one cache carries the dirty copy and must supply data and later write back to memory.

6.5 Reducing Snooping Interference: Inclusion and Duplicated Tags

Problem:

- Every bus transaction requires snooping (checking tags) in each cache.
- Tag checks can **interfere** with normal processor cache accesses.

Solutions:

1. Duplicate tags:

- Keep a small tag-only structure for snooping.
- Avoid interfering with main tag/data arrays.

2. Use L2 as the snoop target (in multilevel caches):

- Direct snoop requests to the **L2 cache**.
- Require **inclusion property**:
 - Every block present in L1 must also be present in L2.
- Then:
 - Snoop only L2.
 - If L2 has the block:
 - It can coordinate invalidating or updating L1 if needed.
- Sometimes even L2's tags are duplicated for further separation.

The inclusion property is discussed more in the cache chapter.

7. Example Snooping Invalidate Protocol (Write-Back Cache)

A typical basic protocol uses three states per block:

1. **Invalid (I)** – no valid data.
2. **Shared (S)** – block is clean (same as memory) and possibly present in multiple caches.
3. **Modified (M)** – block is dirty and exclusive in this cache.

The full protocol is described via:

- A **table** (Figure 4.5): mapping processor/bus events + current state → actions.
- A **state diagram** (Figures 4.6, 4.7): showing state transitions.

7.1 Processor-Side Requests (Top Half of the Table)

For each block, consider:

- **Read hit:** state = S or M
 - Just return data from cache.
- **Read miss:** state = I (or other block conflict)
 - Generate **read miss** on bus.
 - May need to:
 - Write back current block (on replacement if state = M).
 - Then place new read miss.
- **Write hit:**
 - If state = M:
 - Just write into cache (no bus action).
 - If state = S:
 - Need to obtain exclusive access:
 - Generate **invalidate** on bus.
 - Transition S → M.
 - This is often called an **upgrade** or **ownership miss**:
 - You don't fetch data (you already have it).
 - You only upgrade permission.
- **Write miss:**
 - State = I (or conflict in S/M):
 - Might need to write back old block if M.
 - Then place **write miss** on bus.
 - Fetch the block, get it exclusively, and write.

7.2 Bus-Side Events (Bottom Half of the Table)

Each cache snoops the bus and reacts if it has the block:

- **Read miss on bus:**
 - If snooping cache has the block in S:
 - Usually no special action: memory services the read.
 - If block is in M:
 - Cache **supplies the block** on the bus (write-back to memory or directly to requestor).
 - Changes its state $M \rightarrow S$ (shared).
- **Invalidate on bus:**
 - If block is in S:
 - Invalidate it: $S \rightarrow I$.
 - If block is in M:
 - Depending on protocol variant:
 - Write back data and invalidate, or
 - Transition to I and send updated value.
- **Write miss on bus:**
 - If any cache holds block in S:
 - Invalidate its copy: $S \rightarrow I$.
 - If a cache holds block in M:
 - Writes back data and invalidates: $M \rightarrow I$.

7.3 State Diagrams (Figures 4.6 and 4.7)

Two views:

1. Split diagrams:

- Left side: transitions due to processor requests.
- Right side: transitions due to bus requests.
- States: I, S, M.
- Transitions include:
 - Read miss $\rightarrow I \rightarrow S$ or M
 - Write hit in S $\rightarrow$ invalidate others $\rightarrow S \rightarrow M$
 - Bus invalidate on S or M $\rightarrow S/M \rightarrow I$
 - Bus read miss on M $\rightarrow M \rightarrow S$ and supply data.

2. Combined diagram:

- Merge processor- and bus-induced transitions into a single FSM per block.

Key invariants maintained:

- At any time, a block is either:
 - In S state in one or more caches, and **also clean in memory**, or
 - In M state in exactly one cache, and **memory may be stale**.
- If a processor wants to write:
 - It must first move block to **M** state via bus activity (invalidate or write miss).
- Read miss:
 - If some cache has block in M, it supplies the correct data and transitions M→S.

Most of these states and transitions already exist in a uniprocessor (I, clean, dirty).

Only the **coherence-related transitions** (those reacting to bus events) are new.

7.4 Non-Atomicity Complications

The protocol description so far assumes:

Certain sequences of actions related to a miss or write appear **atomic** (no interleaving).

But in real systems:

- Actions like:
 - Detecting a miss
 - Acquiring bus/network
 - Sending invalidates
 - Receiving responses
 - Updating cache
 are **not atomic**.

In bus-based systems:

- You can approximate atomicity:
 - First, arbitrate and **win the bus**.
 - Hold the bus until all coherence actions (invalidates, write-backs) are complete.
 - Use a **global signal line** to indicate “all invalidates have been received and processed”.
 - Only then release the bus.

In systems **without a bus** (e.g., switch-based, point-to-point):

- Miss handling is **not atomic**.
- Races can occur:
 - Two processors trying to write the same block at nearly the same time.
- The design must:
 - Ensure **exactly one winner** in such a race.

- Provide a mechanism for losers to **restart** their miss after the winner's coherence actions complete.
- This requires:
 - Properties of the interconnection network.
 - Protocol-level logic to detect and resolve conflicts.

The book postpones detailed treatment of these **non-bus snoopy implementations** to Appendix H.

8. Real SMP Implementations and Their Limits

8.1 Easy Small-Scale Snoop-Based SMPs

Modern processors such as:

- Intel **Pentium 4 Xeon**
- AMD **Opteron**

are designed to:

- Support **cache-coherent multiprocessing**.
- Provide an external interface that supports:
 - **Snooping**
 - Direct connection of **2-4 processors**
- They include:
 - Large on-chip caches to reduce bus utilization.
- Result:
 - Very convenient, low-cost 2-4 processor SMPs.

8.2 Limitations of SMP and Snooping

As:

- Number of processors increases, or
- Memory demands per processor increase,

any **centralized resource** becomes a bottleneck:

- **Bus:**
 - Must carry both coherence traffic and normal memory traffic.
- **Single memory unit:**
 - Must answer all requests.

With faster processors:

- The number of processors supportable on:

- a single bus, or
- a single memory has **decreased**.

8.3 Increasing Memory and Interconnect Bandwidth

To increase bandwidth while preserving **uniform access time** (UMA):

Designers use:

- **Multiple buses**
- **Interconnection networks:**
 - Crossbars
 - Small-scale point-to-point networks

Memory is divided into multiple **banks**:

- Multiple processors can access different banks simultaneously.
- Overall effective memory bandwidth increases.
- Still preserve **uniform access latency** (each processor sees the same latency to any address).

Figure 4.8 shows this architecture:

- Multiple processors with caches.
- Multiple memory modules.
- Connected by an **interconnection network**, not a single bus.
- Still a **centralized shared memory** system with UMA.

8.4 The AMD Opteron Hybrid

AMD Opteron represents a point between:

- Traditional snoopy SMP, and
- Directory-based NUMA.

Characteristics:

- Memory is **directly attached** to each dual-core processor chip.
- Up to **four** such chips can be connected (i.e., up to 8 cores).
- Coherence protocol:
 - Uses **point-to-point links** to broadcast invalidations and misses to the other processor chips.
 - Since the links are **not shared** like a bus:
 - Completion of invalidations must be confirmed via **explicit acknowledgments**.
 - This is similar to:
 - Snooping (broadcast to find sharers).

- Directory protocols (ack-driven ordering and completion).

Interestingly:

- Local and remote memory latency are **not very different**.
- The OS can treat the system as roughly **UMA**, even though the memory is physically distributed.

8.5 Snooping Scalability Limits and Directory-Based Future

Even without a central bus, snooping protocols require:

- **Broadcast** of every miss to a potentially shared block to **all caches**.

This leads to:

- Large **coherence traffic** that does not shrink with larger caches.
- As processor core counts and core speeds rise:
 - Coherence broadcast traffic **overwhelms the network**.
 - Each cache's ability to respond to snoops becomes a bottleneck.

Therefore:

- Snooping is fundamentally limited in scalability.
- **Directory-based protocols** (Section 4.4) avoid broadcast by explicitly tracking sharers.
- With many cores per chip, designers are increasingly moving toward **directory-based coherence**.

9. Implementing Snoopy Coherence in Modern Systems

Historically:

- First edition of this book (1990) used:
 - A 30-processor single-bus multiprocessor as its "Putting It All Together" example.
 - Bus bandwidth: ~50 MB/s (enough then, totally inadequate now).

By 1995:

- First cache-coherent multiprocessors with **multiple buses** appeared.
- An appendix explained how to implement snooping in multi-bus systems.

By 2006:

- All multiprocessors with more than two processors use:
 - Interconnects other than a single bus (e.g., crossbars, point-to-point networks).
- Designers must:
 - Implement snoopy coherence **without** the simplifying assumption of a single, serializing bus.

Core challenge:

- **Misses and upgrades are not atomic.**
- Need a mechanism to:
 - Detect write/upgrade misses.
 - Coordinate with other caches and memory.
 - Retrieve the most recent data.
 - Ensure that all invalidations are processed.
 - Serialize races on the same block.

Bus-based systems achieve atomicity by:

- Acquiring the bus.
- Holding it until the entire sequence is complete.
- Using a global signal line to indicate “all invalidations are in progress and will complete before the next miss is handled.”

Non-bus systems use:

- Properties of the interconnect (e.g., ordering guarantees).
- Explicit acknowledgment messages.
- Miss-restart mechanisms for losers in write races.

Details are complex and explained in **Appendix H**.

If you'd like, next I can:

- Draw a cleaner **MESI-only state diagram** from the three-state protocol,
- Or convert this entire section into **exam notes / Q&A style**,
- Or give you **example problems** on coherence and walk through them step-by-step.

Below is a **clean, continuous, well-structured, deeply detailed explanation** of **Section 4.3: Performance of Symmetric Shared-Memory Multiprocessors**, written in an easy-to-read paragraph style with *fewer points and more flowing narration*, while still covering **every single concept, example, and detail** from your text.

4.3 Performance of Symmetric Shared-Memory Multiprocessors — Detailed Explanation

When evaluating the performance of a multiprocessor that uses a **snoopy cache coherence protocol**, we must remember that overall behavior is shaped by **two fundamentally different types of cache**

misses. First, there are the familiar uniprocessor miss components—**compulsory, capacity, and conflict misses**—which arise even without parallel execution. Second, multiprocessors introduce additional misses triggered by **interprocessor communication**, where processors interact through shared-memory. These communication-induced misses come from the coherence mechanism itself, which invalidates cache blocks or forces updates, thereby causing more cache misses later. When processor count, cache sizes, or cache block sizes are changed, both uniprocessor and coherence-related misses can shift in different directions, and the overall system behavior is dictated by the balance between them.

To understand the communication-related misses, we divide them into two categories: **true sharing misses** and **false sharing misses**. True sharing misses occur when multiple processors genuinely share the same data word and must communicate updates. For example, in an invalidation-based protocol, the first time a processor writes to a shared block, it must invalidate the copies held by other processors; later, when another processor tries to read that freshly written data, it will miss and fetch the updated block. These misses reflect actual, meaningful sharing of data between processors.

False sharing, on the other hand, arises not because data is truly shared, but because **multiple unrelated words happen to reside in the same cache block**. If one processor writes to word x_1 , and another processor holds the same block because it previously read word x_2 , the write to x_1 will still invalidate the block in the second processor—even though x_2 is logically independent. When the second processor attempts to read x_2 again, it suffers a miss even though no relevant data changed. This type of miss would not exist if each cache block contained exactly one word. False sharing is therefore a byproduct of the block-based granularity of invalidation.

Illustrative Example of True and False Sharing

Assume a cache block contains two separate words, x_1 and x_2 , and that processors P1 and P2 both have this block in shared state. Consider the following sequence:

1. **P1 writes x_1** – This invalidates P2's copy, so if P2 later accesses the block it will miss. This is a **true sharing miss** because P1 genuinely modified shared data.
2. **P2 reads x_2** – This causes a **false sharing miss**, because P2's block was invalidated by the write to x_1 , even though it wants x_2 , whose value has not changed.
3. **P1 writes x_1 again** – Now the block is in shared state (after P2's read). P1 must upgrade to exclusive state, causing another invalidation but no data transfer. This is a **false sharing miss** since P2 never read x_1 .
4. **P1 writes x_2** – Same reasoning as step 3. Another **false sharing miss**.
5. **P2 reads x_2 , which P1 modified** – This is **true sharing**, since the read needs the new value written by P1.

Although both types of misses appear in real systems, false sharing is particularly harmful because it wastes bandwidth and produces meaningless misses that do not help with data communication.

Commercial Workload Evaluation on a Four-Processor SMP

The performance impact of all this can be seen in real workloads. Measurements were taken using a four-processor AlphaServer 4100 system (or a simulator modeled after it). Each Alpha 21164 processor runs at 300 MHz, issues up to four instructions per cycle, and uses a **three-level cache hierarchy**, which is important for understanding miss patterns:

- **L1:** 8 KB instruction cache and 8 KB data cache, direct-mapped, 32-byte blocks; D-cache is write-through to L2 via a write buffer.
- **L2:** 96 KB unified, 3-way associative, write-back, 32-byte blocks.
- **L3:** 2 MB unified, direct-mapped, write-back, 64-byte blocks.

Latency characteristics:

- L2 hit: 7 cycles
- L3 hit: 21 cycles
- Main memory access: 80 cycles
- Cache-to-cache transfer (exclusive block in another cache): 125 cycles

The workloads tested were:

1. **OLTP (Online Transaction Processing)** based on TPC-B with Oracle 7.3.2. These server processes frequently block for I/O and execute many short operations (high OS involvement).
2. **DSS (Decision Support System)** based on TPC-D with six representative queries. DSS queries involve long scans and analytic operations with comparatively fewer context switches.
3. **AltaVista Web Search**, using a memory-mapped 200-GB index and highly optimized inner loops.

The distribution of CPU time shows that OLTP spends much more time in kernel mode and idle, because it incurs frequent I/O waits, while AltaVista spends nearly all its time in user mode due to excellent tuning and minimal OS interaction.

When examining CPI (cycles per instruction), we see dramatic differences: AltaVista achieves around 1.3 CPI, DSS around 1.6, but OLTP suffers a CPI of about **7.0**, indicating severe memory stalls. A breakdown of execution time shows that OLTP spends a large portion of cycles waiting on L3 and main memory.

Effect of Cache Size on OLTP Performance

Because OLTP stresses the memory hierarchy heavily, the study examines its sensitivity to L3 cache size. When L3 is increased from 1 MB to 8 MB (using two-way associativity), performance initially improves substantially. However, almost all of the benefit is obtained simply by increasing from **1 MB → 2 MB**. Beyond 2 MB, gains flatten out.

Why?

When analyzing the sources of L3 misses, five important contributors appear:

1. Instruction misses
2. Capacity/conflict misses
3. Compulsory misses

4. False sharing misses

5. True sharing misses

Increasing L3 size dramatically reduces instruction and capacity/conflict misses, but **compulsory, false sharing, and true sharing misses do not decrease at all**. At larger L3 sizes (4 MB, 8 MB):

- **True sharing misses dominate**, because they cannot be removed by increasing cache size.
- False sharing remains unchanged as well.
- As a result, miss rate reduction hits a wall beyond 2 MB.

Thus, **multiprocessor-induced misses—not uniprocessor misses—become the bottleneck**.

Effect of Increasing Processor Count

Increasing processor count increases the true sharing activity. With a fixed 2 MB L3 cache, scaling from 1 → 8 processors shows that memory access cycles per instruction rise primarily because:

- True sharing misses increase proportionally with more processors.
- Uniprocessor misses (capacity/conflict, compulsory) do not decrease.

The system therefore experiences higher coherence traffic and more contention, hurting performance.

Effect of Block Size on Miss Rate

Increasing the L3 block size from 32 bytes up to 256 bytes affects four miss components:

- **True sharing misses decrease significantly** (more than 2× improvement), because larger blocks capture more of the truly shared region.
- **Compulsory misses decrease** (expected: larger blocks bring more useful neighbors).
- **Capacity/conflict misses slightly improve**, but not much—implying little spatial locality in OLTP.
- **False sharing nearly doubles**, because larger blocks increase the chance that unrelated variables share the same block.

Surprisingly, instruction miss rate does not drop much, suggesting very **poor spatial locality** in OLTP instruction streams—likely due to short basic blocks and highly specialized database code.

Overall, a 128–256 byte L3 block size seems appropriate for such workloads.

Multiprogrammed and OS-Intensive Workload

A second study used a multiprogrammed workload: two independent parallel-make jobs (Andrew benchmark). This workload is more I/O-heavy and has distinct phases: compute-heavy compilation, library installation, and removal of object files. Large portions of execution time are idle, waiting for disk I/O.

The memory system used for evaluation includes:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

- 32 KB L1 instruction cache, 2-way, 64-byte blocks
- 32 KB L1 data cache, 2-way, 32-byte blocks
- 1 MB unified L2 cache, 2-way, 128-byte blocks
- Main memory with 100-cycle access time
- Disk latency 3 ms

Breaking down execution time on eight processors shows:

- Most time is spent idle due to I/O.
- User processes retire 9× more instructions than the kernel, yet take only 4× as long because the kernel has significantly higher miss rates.
- Synchronization overhead is small.

The OS has significantly worse instruction and data cache behavior than user code. OS misses arise from poor locality, larger code footprint, compulsory misses from initializing pages, and real coherence traffic due to shared OS data structures.

Cache Performance Analysis

Examining L1 data miss rates reveals that increasing cache size helps user processes much more than kernel code. User miss rate drops by nearly a factor of 3 when increasing from 32 KB → 256 KB, whereas kernel miss rate drops by only 1.3×.

Increasing cache block size improves both user and kernel miss rates. The improvement is nearly equal, since a large portion of kernel misses are compulsory or capacity misses.

A deeper breakdown of kernel misses shows:

- Compulsory misses do not change with cache size.
- Capacity/conflict misses drop sharply with larger cache sizes.
- Coherence misses rise slightly because larger caches retain blocks longer, increasing the chance that an invalidation will cause a miss.
- Larger block sizes drastically reduce compulsory misses and moderately reduce capacity misses.
- Coherence misses remain roughly unchanged, with false sharing not rising significantly.

However, when measuring **bytes transferred per data reference**, kernel traffic grows substantially with larger blocks. For example, going from 16-byte to 128-byte blocks reduces miss rate by a factor of 3.7, but each miss transfers 8× more data, increasing total traffic by roughly 2×. User-level traffic also increases but starts at a lower baseline.

This demonstrates that OS workloads put far higher pressure on the memory system than user tasks do, especially as block size grows.

To improve performance, operating systems may need **cache-aware memory management**, such as avoiding unnecessary coherence events when reusing memory or providing compiler hints (like those used in IBM Power architectures). Such optimizations are challenging but could yield significant performance benefits.

If you want next:

I can produce:

- A **short revision summary**
- A **full diagram-based conceptual map**
- A **Q&A or exam-style explanation**
- A **comparison between OLTP, DSS, OS workloads** in table form

Just tell me what format you want next.

This part of the chapter is basically answering one big question:

“Snooping is simple and cheap, but it doesn’t scale. So what’s the alternative, and how does it work in detail?”

That alternative is **directory-based coherence** in **distributed shared-memory (DSM)** systems. I’ll walk through this section in a continuous, structured way, but without breaking everything into too many tiny bullet points.

1. Why Snooping Doesn’t Scale and What We Need Instead

Recall from Section 4.2: in a **snooping protocol**, every cache **hears about every cache miss** that might involve shared data. When any core has a read or write miss to a shared block, it sends a transaction on a broadcast medium (like a shared bus), and all caches snoop that transaction. This is cheap and elegant for a small system, because you don’t need any centralized structure to track which cache has what; you just broadcast and let everyone react.

That lack of a central structure is both a **strength** and a **weakness**:

- Strength: the protocol is relatively simple and inexpensive.
- Weakness: it **relies on broadcast** to all caches, which becomes impossible (or too expensive) as the number of processors grows and as memory traffic increases.

The text gives a concrete illustration. Consider a 16-processor system, each with a 512 KB data cache, 64-byte blocks, and a processor core capable of sustaining one data reference per clock at 4 GHz. That’s four data references per nanosecond, which is within reach of a modern superscalar processor with non-blocking caches. If you run real scientific/technical workloads (like those in Appendix H) on such a system, the total bus bandwidth demand caused by the cache traffic (ignoring stalls) can range from around **4 GB/s up to 170 GB/s**. Meanwhile, a real 16-way centralized shared-memory machine in 2006 only provided about **2.4 GB/s of memory bandwidth per processor**, and distributed-memory systems of that time could give something like **12 GB/s per processor** to local memory. In other words, the broadcast requirement of snooping would very quickly saturate the available bandwidth.

We can increase raw memory bandwidth and interconnect bandwidth by **distributing memory** across nodes (as in Figure 4.2): each node has its own local memory and interface to an interconnection network. That naturally separates **local** and **remote** traffic. But if we still rely on **broadcast coherence** on every shared cache miss, we haven't solved the fundamental bottleneck; the coherence traffic itself will kill scalability. So if distributed memory is going to help, we must **eliminate broadcast on every miss**.

This is where **directory-based coherence** comes in.

2. Directory-Based Coherence: The Big Idea

Instead of having every cache listen to every miss, a **directory protocol** maintains an explicit data structure that records, for each memory block:

- The coherence state of the block (e.g., shared, modified).
- Which processors currently have a copy.
- Who owns the modified copy if it's dirty (exclusive).

When a cache has a miss, it sends a **point-to-point message** to the block's "home" node (where the directory for that block lives), and the directory:

- Looks up who has the block and in what state.
- Possibly sends messages to just those specific nodes.
- Updates the directory state accordingly.

Because each block's sharing state is stored in **one known location (its directory entry)**, the system can maintain coherence **without broadcasting to every cache**.

Directory protocols can also be used in centralized shared-memory systems (SMPs) to reduce bandwidth demands. Sun's T1 is given as an example of a multicore system using a directory even though it has a centralized memory.

3. How the Directory Is Organized and Scaled

The simplest (and most common in small to medium systems) directory design is:

- For every physical memory block (of size equal to L2/L3 cache block), there is a **directory entry**.
- Each entry stores:
 - The block's coherence state (uncached, shared, modified).
 - A **bit vector** with one bit per processor indicating which caches have that block.

This is proportional to:

$$\text{number_of_memory_blocks} \times \text{number_of_processors}$$

For multiprocessors with fewer than roughly 200 processors and reasonably sized blocks, this overhead is acceptable. For larger systems, it becomes problematic, so designers adopt tricks like:

- Only keeping directory info for blocks that are actually cached, not for all memory.
- Compressing the sharing information (e.g., group several processors under one bit).

To avoid making the directory itself a bottleneck, it is **distributed** along with memory:

- The physical address space is partitioned across nodes.
- Each node holds the memory and directory entries for its portion of the address space.
- Different directory requests go to different nodes, so directory lookups can be parallel.

The key property we keep is: **for every block, there is exactly one “home” directory entry**, so the directory always has a single authoritative view of who has the block.

Figure 4.19 shows this: each node has a processor + cache, local memory, I/O, and a **directory**.

Directories can be connected to memory via a local bus, dedicated port, or combined into a central node controller for that node.

4. Nodes and Messages in a Directory-Based System

To describe protocol behavior, the text introduces three node roles:

- **Local node** – where the cache miss originates (the requester).
- **Home node** – where the memory (and directory entry) for that address resides.
- **Remote node** – any other node that currently has a cached copy, either shared or exclusive.

Local and home can be the same node. Remote can also coincide with local or home in certain cases; the protocol logic stays conceptually the same, but some network messages reduce to local actions.

The protocol is implemented as **message exchanges** among these nodes. Figure 4.20 lists the message types and their purposes. The key messages are:

- From local cache to home directory:
 - **Read miss**: “Processor P has a read miss at address A; send the data and mark P as a sharer.”
 - **Write miss**: “Processor P has a write miss at A; send the data and make P the exclusive owner.”
 - **Invalidate**: “Send invalidations to all caches that hold A.”
- From home directory to remote caches:
 - **Invalidate**: “Invalidate your copy of block A.”
 - **Fetch**: “Send the block A back (e.g., for a read) and downgrade your state to shared.”
 - **Fetch/Invalidate**: “Send the block A back and invalidate your copy” (for write misses that need the latest data and exclusive access).
- From home/remote to caches:
 - **Data value reply**: directory (or memory) → cache with the data value D.
 - **Data write back**: cache → home directory when evicting a modified block or responding to a fetch/fetch-invalidate.

The protocol assumes messages are received and processed **in the order they are sent**. This simplifying assumption is used in this section to keep the logic clear, but real networks can reorder messages, and so real implementations must handle out-of-order arrivals and use mechanisms like **acknowledgements** to ensure that invalidations complete and that write ordering is preserved. These additional complications (serialization of writes, ensuring invalidates finished, handling network reordering) are discussed in more detail in Appendix H and in the section on memory consistency (4.6).

5. Basic Directory States and Conceptual Behavior

At a high level, a **directory-based coherence protocol** has the same semantic states as a snooping protocol, but split between:

- State seen in each **cache** for each cached block.
- State stored in the **directory** for each memory block.

The directory's view of each memory block can be:

- **Uncached** – no processor caches this block.
- **Shared** – one or more processors have a cached copy; memory holds a **clean, up-to-date** value.
- **Modified (Exclusive)** – exactly one processor (the owner) has the block; it has been written and memory is **stale**.

In addition, the directory maintains the set **Sharers**, which records which processors have the block. When the block is in Modified state, Sharers contains exactly the owner's identity. When it is Shared, Sharers contains all readers. When Uncached, Sharers is empty.

At the **cache side**, the states and transitions look almost the same as the snooping protocol: Invalid, Shared(read-only), Modified(read/write). The main difference is that instead of broadcasting writes and misses on a bus, a cache sends point-to-point messages to the home directory, and the directory then sends targeted messages to other caches as needed.

6. Cache-Side Protocol: Figure 4.21

Figure 4.21 shows the state transitions for a single cache block under a directory protocol. The states are:

- **Invalid**: no valid data.
- **Shared (read-only)**: block is present and clean; may be present in multiple caches.
- **Modified (read/write)**: block is present, dirty, and exclusive in this cache.

Transitions are triggered either by:

- Local processor actions (read miss, write miss, read hit, write hit).
- Directory messages (data replies, invalidates, fetch/fetch-invalidate requests).

Conceptually, behavior is:

- A **read miss** in Invalid state:
 - Cache sends a read-miss message to the home directory.
 - Waits for a **data value reply**.
 - On reply, the block enters Shared (unless the protocol is optimized for Exclusive on first read and no other sharers).
- A **write miss** in Invalid state:
 - Cache sends a write-miss message to the directory.
 - Directory arranges for:
 - Retrieval of the latest value (from memory or owner).
 - Invalidations of any sharers.
 - Cache enters Modified when the data and ownership are granted.
- A **write hit** in Shared state:
 - The cache cannot just write; it needs exclusive ownership.
 - It sends a write-miss (or upgrade/ownership) request to the directory.
 - The directory invalidates all other sharers.
 - Cache transitions to Modified.
- An **Invalidate** message from the directory:
 - If cache holds the block in Shared or Modified, it invalidates it.
 - If in Modified, it may also have to **write back** the block before invalidating (depending on protocol variant).
- A **Fetch** or **Fetch/Invalidate** from the directory:
 - Cache sends back the data value (write-back).
 - Transitions to Shared (Fetch) or Invalid (Fetch/Invalidate).

The conceptual rule remains the same: any block must be **exclusive** (Modified) in exactly one cache before it can be written, and any shared block must be **up to date in memory**.

7. Directory-Side Protocol: Figure 4.22

The directory maintains the global view of each memory block. Its states mirror the cache states:

- **Uncached** – no cache holds the block.
- **Shared** – one or more caches have clean copies; memory is up to date.
- **Exclusive (Modified)** – one cache holds the latest, dirty copy; memory is stale.

Additionally, it maintains **Sharers**, a set of processors that have the block.

For each directory state, there are specific reactions to incoming messages:

When the block is Uncached

- **Read miss** from some processor P:
 - Memory has the correct data.
 - Send data from memory to P with a Data value reply.
 - Set state to Shared.
 - Set **Sharers** = {P}.
- **Write miss** from P:
 - Memory has the correct data.
 - Send data from memory to P.
 - Set state to Exclusive.
 - Set **Sharers** = {P} (P is now the owner).

When the block is Shared

- Memory is still up to date with the latest value.
- **Read miss** from P:
 - Send data from memory to P.
 - Add P to the Sharers set: **Sharers** = **Sharers** $\cup$ {P}.
 - State remains Shared.
- **Write miss** from P:
 - Need to give P exclusive ownership.
 - Send **Invalidate** messages to all processors in Sharers.
 - Once invalidations are acknowledged (in a real system), set **Sharers** = {P}.
 - State becomes Exclusive.
 - Data can be supplied from memory (since it is up to date).

When the block is Exclusive

- Memory is out of date; the **owner** has the correct copy.
- Sharers = {Owner}.

Three types of directory requests are now possible:

1. **Read miss** from some processor P (not the owner):
 - Directory sends a **Fetch** message to the owner:
 - Owner writes back the block (Data write back) and downgrades its cache state to Shared.
 - Directory stores the returned value in memory.
 - Directory then sends a Data value reply to P.
 - P now has a Shared copy.
 - Sharers now includes both the former owner and P.

- State changes from Exclusive → Shared.

2. **Data write back** from the owner:

- This happens when the owner evicts the block.
- Directory writes the block to memory (which now becomes correct).
- State becomes Uncached.
- Sharers set becomes empty.

3. **Write miss** from some other processor Q:

- Q wants ownership.
- Directory sends a **Fetch/Invalidate** message to the current owner.
 - Owner sends back the block data and invalidates it from its cache.
- Directory forwards the data to Q in a Data value reply.
- Q becomes the new owner: **Sharers** = {Q}.
- State remains Exclusive.

Note how similar this is to the snooping protocol logic, but instead of everyone watching a bus, the directory coordinates point-to-point messages based on its Sharers set.

8. Practical Complications and Optimizations

The diagrams and descriptions are conceptually clean but assume certain actions are **atomic** (e.g., getting data from owner and sending to requester as one conceptual step). Real systems must handle:

- Non-atomic memory and directory operations.
- Network reordering and delays.
- Potential deadlocks and race conditions.
- Ensuring invalidations complete before a write is considered finished.

These issues are the same kind of complications we saw when moving snoop protocols off a single bus and onto more complex networks. Directory protocols require explicit **acknowledgements** to know when invalidations are done and to enforce write ordering. The “full” implementation details live in Appendix H.

Real implementations also add **optimizations**. For example, in the basic description, when a miss occurs to an Exclusive block, the owner always sends the block back to the **home directory**, which then sends it to the requester and writes it to memory. Real machines often optimize this by having the owner **forward** the data directly to the requester (while also informing the directory), reducing latency. These optimizations, however, increase complexity and can introduce more chances for deadlock, so they must be designed carefully.

If you'd like, I can next:

- Summarize the differences between **snooping** and **directory** protocols in a compact comparison, or

- Turn this into exam-style Q&A, or
- Draw a mental “flowchart” for read miss / write miss handling in both schemes.

This section is shifting from “**do caches agree on the latest values?**” (coherence) to “**when are those new values allowed to be seen by others?**” (consistency). Both are different but related.

I’ll go through it in a continuous, structured way, but mostly in paragraphs so it’s easier to read and remember.

1. Coherence vs Consistency – What’s the New Question?

Cache coherence guarantees that, for each individual memory location, all processors eventually agree on a single, consistent value. For example, if multiple cores read and write variable *x*, coherence ensures that they don’t end up stuck forever with conflicting values (e.g., some see 1 and others see 2 indefinitely).

But coherence **does not specify when** a new value becomes visible to other processors, especially across **different variables**. That’s a **memory consistency** question.

Rephrased:

- Coherence: “If everyone reads *X* long enough, will they all agree on the same sequence of values?”
- Consistency: “In what **order** are writes by one processor observed by other processors, especially when multiple variables are involved?”

Because processors communicate via **shared variables**—both for normal data and for synchronization (locks, flags, barriers)—we care about **how reads and writes to different locations are ordered across processors**.

So the core question becomes:

What rules must a multiprocessor obey about **the order in which reads and writes to different addresses become visible** to different processors?

2. Why This Is Tricky: The A/B Example

Consider two processes, **P1** and **P2**, running on different processors:

```

text

P1:          P2:
A = 0;       B = 0;
...         ...
A = 1;       B = 1;
L1: if (B == 0) ...  L2: if (A == 0) ...

```

Assumptions:

- Variables **A** and **B** start at 0.
- Both processors have cached copies of **A** and **B** initially (value 0).

Intuitively, if:

- **P1** does **A = 1** before **L1**, and
- **P2** does **B = 1** before **L2**,

then you might think it's **impossible** for both:

- **if (B == 0)** and
- **if (A == 0)**

to be true simultaneously, since one of them should have seen the other's write.

But that rests on an assumption: that as soon as a write happens, all other processors "instantly" see the new value, or that any invalidations are processed before further loads.

In reality:

- Write invalidations may be **delayed**.
- Writes may sit in store buffers.
- Processors might continue executing and do reads before the invalidations take effect.

So it is possible, on some machines, that:

- **P1** writes **A = 1**, but still reads **B == 0** because it hasn't yet seen **P2**'s update to **B**.
- **P2** writes **B = 1**, but still reads **A == 0** for the same reason.

Then both **if** conditions can be true. The question is: **is this behavior allowed?** If yes, under what rules? If no, what must the hardware guarantee?

This is exactly what **memory consistency models** formalize.

3. Sequential Consistency – The Simple but Strict Model

The most intuitive and widely taught model is **sequential consistency (SC)**.

Definition (informal):

The execution result is as if:

1. Each processor's memory operations (loads and stores) occur **in program order**, and
2. All processors' operations are **interleaved** in some global order, but that global order respects each processor's individual program order.

In other words, you can imagine all operations from all processors being merged into a single sequence of memory operations, where:

- Each processor's operations appear in the order they occur in its own program.
- The result of the execution matches this merged sequence.

Under sequential consistency:

- The assignments **A = 1** and **B = 1** must **appear in the global order** before their respective **if** statements.
- That prevents weird cases where both **if (B == 0)** and **if (A == 0)** evaluate to true—because at least one of them must see the other's write in the global interleaving.

To implement SC naively, we could enforce **very strict ordering**:

- A processor does **not complete** a memory operation (especially a write) until:
 - All coherence actions (e.g., invalidations) caused by that operation have been fully processed by all relevant caches.
- Or equivalently, it does not start the **next** memory operation until the previous one is globally visible.

This means that operations like:

```
text

write to B
read from A
```

cannot overlap; the read must wait until the write (and its invalidations) are fully complete. That's the price of SC.

Also note: consistency rules involve **different locations**. In the example, we are ordering a write to **A** with a read of **B**. So we can't just treat them as independent.

4. Performance Cost of Sequential Consistency – Example

Now consider the performance implications on a real machine.

We're given a hypothetical system where:

- A **write miss** (i.e., writing to a block not in exclusive state) takes:
 - 50 cycles to establish ownership of the block.
- Once ownership is established:
 - Each invalidate message costs 10 cycles to issue.
 - An invalidate takes 80 cycles to complete and be acknowledged.
- There are 4 other processors that share this cache block, so 4 invalidates are needed.

Under **sequential consistency**, the writer must wait until:

- It gains ownership.
- It sends invalidations to all sharers.
- All invalidations complete (acknowledged).

Let's compute:

- Ownership time: 50 cycles.
- Invalidation issue:
 - First invalidate issued right after ownership is gained.
 - Next 3 invalidates at intervals of 10 cycles.
 - The last invalidate is issued **40 cycles after** ownership is gained ($10 + 10 + 10 + 10$).
- The last invalidate takes 80 cycles to complete after it is issued.

So total stall = ownership time + "issue delay" for last invalidate + invalidate completion time
 $= 50 + 40 + 80$
 $=$ **170 cycles** per write.

If we **relax** SC and allow the processor to continue after just gaining ownership (not waiting for invalidates to finish), then the stall per write could be only **50 cycles**, or even less with a smart write buffer.

So strict SC can introduce very large stalls, especially in large systems with many sharers and high network latency.

5. Two Directions: Better Implementation vs Relaxed Models

Because SC is simple for programmers but costly for performance, architects explored two directions:

1. Try to keep SC but hide latency

Use techniques like speculation, out-of-order execution, prefetching, and clever write-buffering to overlap long-latency coherence actions with useful work. This is discussed later in Section 4.7.

2. Relax the consistency model

Loosen the strict ordering rules so hardware can reorder more, but use **synchronization operations** to keep programs correct. This leads to **relaxed memory consistency models**, which we'll discuss in this section.

Before we dive into relaxed models, we need to understand what programmers typically assume.

6. The Programmer's View: Synchronized, Data-Race-Free Programs

Although SC is easy to reason about, hardware rarely implements pure SC in the simplest way because of the performance cost. However, many programs don't actually rely on the most chaotic interleavings; instead, they use **synchronization** (locks, barriers, semaphores, atomic operations) to coordinate access to shared variables.

We say a program is **synchronized** if:

Every access (read or write) to shared data is **ordered by synchronization operations**.

More precisely:

- For any shared variable:
 - A write by one processor and a subsequent access (read or write) by another processor are always separated by:
 - A synchronization operation after the write, and
 - A synchronization operation before the access by the other processor.

When this is true, the program is **data-race-free**:

- There is no “racing” between unsynchronized accesses to the same variable.
- The result of the program does not depend on unpredictable timing.

Example:

- Suppose two processors both read and update a shared variable `x`.
- They protect access with a **lock**:
 - Processor 1: `lock(L); x = x + 1; unlock(L);`
 - Processor 2: `lock(L); x = x + 1; unlock(L);`
- Any read or write of `x` by one processor is separated from any access to `x` by the other processor by:
 - `unlock` (after the write), and
 - `lock` (before the other processor accesses).

So all accesses to `x` are ordered by synchronization operations. This is a **synchronized** use of shared data.

If instead two processors modify `x` without locks or any synchronization, then which write “wins” depends on timing—this is a **data race**, and program behavior becomes unpredictable and almost impossible to reason about, even under SC.

Practically:

- Most well-written parallel programs are **synchronized**.
- Programmers avoid constructing their own low-level synchronization from random loads and stores because:
 - It is extremely error-prone.
 - It may break under future hardware implementations.
- They instead rely on **standard synchronization libraries** (pthread mutexes, language-level atomics, etc.), which are implemented carefully for the architecture.

Crucially, if the hardware and compiler respect the semantics of these synchronization operations, a **synchronized program will behave as if the machine were sequentially consistent**, even if the underlying hardware uses a more relaxed model.

7. Relaxed Consistency Models: Core Idea

The key idea behind **relaxed consistency models** is:

Allow ordinary reads and writes to complete **out of order** (for performance), but enforce strict ordering around **synchronization operations**, so that all properly synchronized (data-race-free) programs see behavior equivalent to sequential consistency.

To describe these models more formally, we think in terms of ordering rules like:

- **R → W**: a read must complete before a subsequent write.
- **R → R**: a read must complete before a subsequent read.
- **W → R**: a write must complete before a subsequent read.
- **W → W**: a write must complete before a subsequent write.

Sequential consistency maintains **all four** orderings for all memory operations.

Relaxed models are defined by **which of these orderings they allow to be broken** for ordinary memory operations, with synchronization operations acting as “fences” that re-impose ordering when needed.

Some classic examples:

1. Relaxing **W → R**:

This gives **Total Store Ordering (TSO)** or **Processor Consistency**.

Writes by a single processor are still seen in order by everyone, but a processor might be allowed to perform a following read before a prior write has fully completed globally. Many programs that work under SC still work under TSO without extra effort, since writes remain ordered.

2. Relaxing **W → W**:

This gives **Partial Store Order (PSO)**.

Even the order of writes from the same processor might be relaxed, as long as synchronization operations enforce order when needed.

3. Relaxing **R → W** and **R → R**:

This leads to models like:

- **Weak Ordering**
- **PowerPC consistency model**
- **Release Consistency**

These models allow extensive reordering of reads with each other and with writes, as long as synchronization primitives (locks, barriers, “acquire/release” operations) enforce necessary ordering constraints.

Under all these relaxed models, the system must still ensure that:

- **Synchronization operations are not reordered past each other**, or at least not in ways that break their semantics.
- Properly synchronized programs (data-race-free) see behavior that is effectively SC.

The advantage is that the hardware can do aggressive optimizations like:

- Letting reads pass writes in a store buffer.
- Reordering loads and stores around each other.
- Overlapping invalidations and coherence actions with other useful work.

The downside is that the precise rules become harder to explain and reason about, which is why the details are handled by architects, compiler writers, and synchronization library implementers rather than by most application programmers.

8. Final Remarks: What Do Real Systems Do?

Most modern multiprocessors implement **some form of relaxed consistency**, not strict SC, for performance reasons:

- Some implement something like **processor consistency / TSO** (e.g., x86-like models).
- Others implement **weak ordering** or **release consistency** (common in RISC and NUMA systems).

However:

- Most programmers write **synchronized programs**, using standard synchronization primitives.
- Synchronization libraries are carefully implemented to ensure that these primitives enforce the correct ordering for the target consistency model.
- As a result, correctly synchronized programs behave **as if** the hardware were sequentially consistent.

There is also an alternative viewpoint discussed later: with powerful **speculation**, out-of-order execution, and rollback, it may be possible to get **much of the performance benefit** of relaxed consistency even if the architecture exposes a simpler, stronger model like sequential consistency or processor consistency. In that debate, compilers and their ability to optimize memory accesses to potentially shared variables also play a major role.

So, in practice:

- Hardware uses relaxed models for speed.
- Synchronization libraries and compilers hide the complexity.
- Programmers are encouraged to write **data-race-free** code and think in terms of SC-like behavior around their synchronized data.

If you want, next I can:

- Give a compact “exam-summary” of SC vs TSO vs weak ordering vs release consistency, or
- Turn the A/B example into a few multiple-choice style reasoning questions, or
- Explain how specific real-world architectures (like x86 or ARM) fit into these models.